

LSTM#

<https://pytorch.org/docs/stable/generated/torch.nn.LSTM.html#torch.nn.LSTM>

Description:

Apply a multi-layer long short-term memory (LSTM) RNN to an input sequence. For each element in the input sequence, each layer computes the following function: where h_t is the hidden state at time t , c_t is the cell state at time t , x_t is the input at time t , h_{t-1} is the hidden state of the layer at time $t-1$ or the initial hidden state at time 0, and i_t , f_t , g_t , o_t are the input, forget, cell, and output gates, respectively. σ is the sigmoid function, and $\odot$ is the Hadamard product. In a multilayer LSTM, the input $x^{(l)}$ of the l -th layer ($l \geq 2$) is the hidden state $h^{(l-1)}$ of the previous layer multiplied by dropout $\delta^{(l-1)}$ where each $\delta^{(l-1)}$ is a Bernoulli random variable which is 0 or 1 with probability dropout. If `proj_size > 0` is specified, LSTM with projections will be used. This changes the LSTM cell in the following way. First, the dimension of h_t will be changed from `hidden_size` to `proj_size` (dimensions of W_{hi} will be changed accordingly). Second, the output hidden state of each layer will be multiplied

Function Signature

```
class torch.nn.LSTM(input_size, hidden_size, num_layers=1,
bias=True, batch_first=False, dropout=0.0,
bidirectional=False, proj_size=0, device=None, dtype=None)
[source]#
```

Parameters

Inputs: input, (h_0, c_0)

Outputs: output, (h_n, c_n)

Variables

Code Examples

```
>>> rnn = nn.LSTM(10, 20, 2)
>>> input = torch.randn(5, 3, 10)
>>> h0 = torch.randn(2, 3, 20)
>>> c0 = torch.randn(2, 3, 20)
>>> output, (hn, cn) = rnn(input, (h0, c0))
```

Notes & Warnings

NOTE Note All the weights and biases are initialized from $U(-k,k)\sqrt{k}$ where $k = \frac{1}{\text{hidden_size}}$

NOTE Note For bidirectional LSTMs, forward and backward are directions 0 and 1 respectively. Example of splitting the output layers when `batch_first=False`: `output.view(seq_len, batch, num_directions, hidden_size)`.

NOTE Note For bidirectional LSTMs, `h_n` is not equivalent to the last element of output; the former contains the final forward and reverse hidden states, while the latter contains the final forward hidden state and the initial reverse hidden state.

NOTE Note `batch_first` argument is ignored for unbatched inputs.

NOTE Note `proj_size` should be smaller than `hidden_size`.

⚠ WARNING

Warning There are known non-determinism issues for RNN functions on some versions of cuDNN and CUDA. You can enforce deterministic behavior by setting the following environment variables: On CUDA 10.1, set environment variable `CUDA_LAUNCH_BLOCKING=1`. This may affect performance. On CUDA 10.2 or later, set environment variable (note the leading colon symbol) `CUBLAS_WORKSPACE_CONFIG=:16:8` or `CUBLAS_WORKSPACE_CONFIG=:4096:2`. See the cuDNN 8 Release Notes for more information.

NOTE

Note If the following conditions are satisfied: 1) cudnn is enabled, 2) input data is on the GPU 3) input data has dtype `torch.float16` 4) V100 GPU is used, 5) input data is not in PackedSequence format persistent algorithm can be selected to improve performance.

Detailed Documentation

Documentation

Apply a multi-layer long short-term memory (LSTM) RNN to an input sequence. For each element in the input sequence, each layer computes the following function:

where $h_{t,t}$ is the hidden state at time t , $c_{t,t}$ is the cell state at time t , $x_{t,t}$ is the input at time t , h_{t-1} is the hidden state of the layer at time $t-1$ or the initial hidden state at time 0, and i_t , f_t , g_t , o_t are the input, forget, cell, and output gates, respectively. σ is the sigmoid function, and $\odot$ is the Hadamard product.

In a multilayer LSTM, the input $x^{(l)}$ of the l -th layer ($l \geq 2$) is the hidden state $h^{(l-1)}$ of the previous layer multiplied by dropout $\delta^{(l-1)}$ where each $\delta^{(l-1)}$ is a Bernoulli random variable which is 000 with probability dropout.

If `proj_size > 0` is specified, LSTM with projections will be used. This changes the LSTM cell in the following way. First, the dimension of `h_t` will be changed from `hidden_size` to `proj_size` (dimensions of W_{hi} will be changed accordingly). Second, the output hidden state of each layer will be multiplied by a learnable projection matrix: $h_t = W_{hr} h_t$. Note that as a consequence of this, the output of LSTM network will be of different shape as well. See Inputs/Outputs sections below for exact dimensions of all variables. You can find more details in <https://arxiv.org/abs/1402.1128>.

`input_size` – The number of expected features in the input x

`hidden_size` – The number of features in the hidden state h

`num_layers` – Number of recurrent layers. E.g., setting `num_layers=2` would mean stacking two LSTMs together to form a stacked LSTM, with the second LSTM taking in outputs of the first LSTM and computing the final results. Default: 1

`bias` – If False, then the layer does not use bias weights b_{ih} and b_{hh} . Default: True

`batch_first` – If True, then the input and output tensors are provided as (batch, seq, feature) instead of (seq, batch, feature). Note that this does not apply to hidden or cell states. See the Inputs/Outputs sections below for details. Default: False

`dropout` – If non-zero, introduces a Dropout layer on the outputs of each LSTM layer except the last layer, with dropout probability equal to dropout. Default: 0

`bidirectional` – If True, becomes a bidirectional LSTM. Default: False

proj_size – If > 0 , will use LSTM with projections of corresponding size. Default: 0

input: tensor of shape (L, H_{in}) for unbatched input, (L, N, H_{in}) when `batch_first=False` or (N, L, H_{in}) when `batch_first=True` containing the features of the input sequence. The input can also be a packed variable length sequence. See `torch.nn.utils.rnn.pack_padded_sequence()` or `torch.nn.utils.rnn.pack_sequence()` for details.

h_0: tensor of shape $(D * \text{num_layers}, H_{out})$ for unbatched input or $(D * \text{num_layers}, N, H_{out})$ containing the initial hidden state for each element in the input sequence. Defaults to zeros if (h_0, c_0) is not provided.

c_0: tensor of shape $(D * \text{num_layers}, H_{cell})$ for unbatched input or $(D * \text{num_layers}, N, H_{cell})$ containing the initial cell state for each element in the input sequence. Defaults to zeros if (h_0, c_0) is not provided.

output: tensor of shape $(L, D * H_{out})$ for unbatched input, $(L, N, D * H_{out})$ when `batch_first=False` or $(N, L, D * H_{out})$ when `batch_first=True` containing the output features (h_t) from the last layer of the LSTM, for each t . If a `torch.nn.utils.rnn.PackedSequence` has been given as the input, the output will also be a packed sequence. When `bidirectional=True`, output will contain a concatenation of the forward and reverse hidden states at each time step in the sequence.

h_n: tensor of shape $(D * \text{num_layers}, H_{out})$ for unbatched input or $(D * \text{num_layers}, N, H_{out})$ containing the final hidden state for each element in the sequence. When `bidirectional=True`, h_n will contain a concatenation of the final forward and reverse hidden states, respectively.

`c_n`: tensor of shape $(D \times \text{num_layers}, H_{\text{cell}})$ ($D \times \text{num_layers}$, H_{cell}) for unbatched input or $(D \times \text{num_layers}, N, H_{\text{cell}})$ ($D \times \text{num_layers}$, N , H_{cell}) containing the final cell state for each element in the sequence. When `bidirectional=True`, `c_n` will contain a concatenation of the final forward and reverse cell states, respectively.

`weight_ih_l[k]` – the learnable input-hidden weights of the k^{th} layer ($W_{ii}|W_{if}|W_{ig}|W_{io}$), of shape $(4 \times \text{hidden_size}, \text{input_size})$ for $k = 0$. Otherwise, the shape is $(4 \times \text{hidden_size}, \text{num_directions} \times \text{hidden_size})$. If `proj_size > 0` was specified, the shape will be $(4 \times \text{hidden_size}, \text{num_directions} \times \text{proj_size})$ for $k > 0$

`weight_hh_l[k]` – the learnable hidden-hidden weights of the k^{th} layer ($W_{hi}|W_{hf}|W_{hg}|W_{ho}$), of shape $(4 \times \text{hidden_size}, \text{hidden_size})$. If `proj_size > 0` was specified, the shape will be $(4 \times \text{hidden_size}, \text{proj_size})$.

`bias_ih_l[k]` – the learnable input-hidden bias of the k^{th} layer ($b_{ii}|b_{if}|b_{ig}|b_{io}$), of shape $(4 \times \text{hidden_size})$

`bias_hh_l[k]` – the learnable hidden-hidden bias of the k^{th} layer ($b_{hi}|b_{hf}|b_{hg}|b_{ho}$), of shape $(4 \times \text{hidden_size})$

`weight_hr_l[k]` – the learnable projection weights of the k^{th} layer of shape $(\text{proj_size}, \text{hidden_size})$. Only present when `proj_size > 0` was specified.

`weight_ih_l[k]_reverse` – Analogous to `weight_ih_l[k]` for the reverse direction. Only present when `bidirectional=True`.

`weight_hh_l[k]_reverse` – Analogous to `weight_hh_l[k]` for the reverse direction. Only present when `bidirectional=True`.

`bias_ih_l[k]_reverse` – Analogous to `bias_ih_l[k]` for the reverse direction. Only present when `bidirectional=True`.

`bias_hh_l[k]_reverse` – Analogous to `bias_hh_l[k]` for the reverse direction. Only present when `bidirectional=True`.

`weight_hr_l[k]_reverse` – Analogous to `weight_hr_l[k]` for the reverse direction. Only present when `bidirectional=True` and `proj_size > 0` was specified.

All the weights and biases are initialized from $U(-k, k) \setminus \mathcal{U}(-\sqrt{k}, \sqrt{k}) \cup (-k, k)$ where $k = \frac{1}{\text{hidden_size}}$ $k = \frac{1}{\text{hidden_size}}$

For bidirectional LSTMs, forward and backward are directions 0 and 1 respectively. Example of splitting the output layers when `batch_first=False`: `output.view(seq_len, batch, num_directions, hidden_size)`.

For bidirectional LSTMs, `h_n` is not equivalent to the last element of output; the former contains the final forward and reverse hidden states, while the latter contains the final forward hidden state and the initial reverse hidden state.

`batch_first` argument is ignored for unbatched inputs.

`proj_size` should be smaller than `hidden_size`.

There are known non-determinism issues for RNN functions on some versions of cuDNN and CUDA. You can enforce deterministic behavior by setting the following environment variables:

On CUDA 10.1, set environment variable `CUDA_LAUNCH_BLOCKING=1`. This may affect performance.

On CUDA 10.2 or later, set environment variable (note the leading colon symbol) `CUBLAS_WORKSPACE_CONFIG=:16:8` or `CUBLAS_WORKSPACE_CONFIG=:4096:2`.

See the cuDNN 8 Release Notes for more information.

If the following conditions are satisfied: 1) cudnn is enabled, 2) input data is on the GPU 3) input data has dtype torch.float16 4) V100 GPU is used, 5) input data is not in PackedSequence format persistent algorithm can be selected to improve performance.